Hybrid Ant-Colony Metaheuristics for QoS-Aware Traffic-Engineering Optimization in Software-Defined Networks

João Paulo Versiani Ladeia, and Nilton Alves Maia

***Abstract*—The relentless growth of Internet users, devices and multimedia traffic makes pure infrastructure investment economically unappealing and motivates smarter use of existing resources. Software-Defined Networking (SDN) separates the control and data planes, exposing an ideal substrate on which to implement traffic engineering (TE) at the routing level. This paper formulates the routing of concurrent application flows in an SDN as a combinatorial optimization problem that satisfies Quality-of-Service (QoS) constraints such as end-to-end delay and per-link bandwidth. Two hybrid heuristic strategies, Ant Colony Optimization combined with a Genetic Algorithm (ACO+GA) and with Differential Evolution (ACO+DE), first generate a pool of candidate routes with an AntNet-based colony and then select the best route set according to a weighted objective function. The resulting configurations are compared against a baseline that routes every flow along the Euclidean shortest path (Dijkstra). On a 52-node mesh topology carrying 17 concurrent applications, both hybrids reach the optimal objective value (132) whereas Dijkstra reaches only 120, and discrete-event network simulation shows the optimized network delivers $+37\%$ throughput, $\approx 1.7\times$ lower end-to-end delay, $\approx 45\times$ lower jitter and $\approx 13\times$ lower packet loss with respect to the Dijkstra-routed network. The results confirm that traffic-engineering optimization is an effective, low-cost alternative to capacity expansion in SDNs.**

***Index Terms*—Software-Defined Networking, traffic engineering, QoS routing, Ant Colony Optimization, Genetic Algorithm, Differential Evolution, Dijkstra.**

I. Introduction

THE number of Internet users grew from roughly 400 million in the year 2000 to about 3.6 billion by 2017, and the shift toward bandwidthhungry multimedia applications has profoundly changed the traffic mix that operators must carry [1]. At the same time, network operators face strong pressure to contain capital expenditure. Traffic engineering (TE) — the set of mechanisms that shape, route and dimension network traffic — offers an alternative: instead of *adding* bandwidth, use the *existing* bandwidth more intelligently.

Until recently TE was hard to realize because conventional devices are vertically integrated, closed "black boxes" that run vendor-specific firmware. Software-Defined Networking changes this. By decoupling the control plane from the data plane and exposing a programmable interface (OpenFlow), an SDN controller can install forwarding rules that steer traffic along arbitrary, operator-defined paths [2], [3]. This programmability makes SDN a prime vehicle for TE and route optimization.

Yet the routing that a TE strategy must solve is not trivial. When many applications with different QoS requirements (minimum bandwidth, maximum end-to-end delay, priority) are active simultaneously, a path that is locally shortest may overload shared links and degrade the quality of every flow that uses them. The routing decision becomes a constrained combinatorial optimization: choose, for every application, a route such that all links stay below capacity and every application meets its delay budget, while giving preference to high-priority flows.

This paper proposes and evaluates two hybrid metaheuristics that solve this problem for an SDN:

- **ACO+GA**: an Ant Colony Optimization stage (AntNet) generates a pool of diverse candidate routes; a Genetic Algorithm selects the best set.
- **ACO+DE**: the same ACO stage is followed by Differential Evolution for the route selection.

The optimized route sets are then embedded in a discrete-event network simulation that emulates the forwarding behavior of the SDN, and their performance is benchmarked against a network that routes every flow along the Euclidean shortest path computed by Dijkstra's algorithm.

A. Contributions

The main contributions of this work are:

1) a QoS-aware mathematical model for TE routing in SDN (Section III);
2) two hybrid ACO+GA / ACO+DE optimization frameworks (Section IV);
3) a reproducible discrete-event emulation of the resulting SDN (Section V);
4) an experimental comparison on a realistic 52-node mesh (Section VI).

II. Quality of Service and Software-Defined Networking

A. SDN Architecture

An SDN is logically structured into three layers: the *application*, *control* and *data* planes. The control plane hosts one or more controllers that hold a global view of the network; the data plane is composed of simple forwarding devices (switches); the two are connected through an open southbound

J. P. Versiani Ladeia and N. A. Maia are with the Graduate Program in Computational Modeling and Systems, Universidade Estadual de Montes Claros, Brazil.

TABLE I
APPLICATION DEMANDS ON THE STUDIED TOPOLOGY (17 FLOWS).

App	Type	λ_k (Mbps)	h_k (ms)	δ_k
1	Data	0.256	1000	0
2	Data	0.256	1000	0
3	Video	1.744	150	0
4	Video	1.200	150	5
5	Data	0.256	1000	0
6	Data	0.256	1000	0
7	Data	0.256	1000	0
8	Voice	0.512	150	5
9	Voice	0.512	150	5
10	Data	0.256	1000	0
11	Video	1.200	150	5
12	Video	1.200	150	0
13	Voice	0.512	150	5
14	Data	0.256	1000	0
15	Voice	0.512	150	5
16	Data	0.256	1000	0
17	Data	0.256	1000	0

interface (OpenFlow) and the applications reach the controller through a northbound API. Because the controller dictates the forwarding tables, it can compute and install non-shortest, QoS-compliant routes — exactly the capability that traffic engineering requires.

B. Traffic Engineering for QoS

TE traditionally handles network planning and dimensioning, routing policies, and traffic shaping. In an SDN, TE is lifted to the control plane: a controller-side module computes a set of routes that satisfy QoS constraints and pushes the resulting forwarding entries to the switches, typically during off-peak windows to avoid disturbing live traffic. This is the off-line optimization model adopted here.

III. PROBLEM MODEL

The network is modeled as a directed graph $G = (V, E)$ with $|V| = n$ nodes (switches and end-hosts) and a set of E links. Each link $(i, j) \in E$ carries a capacity μ_{ij} (bandwidth, in Mbps) and a one-way transmission delay c_{ij} (in ms). A set of L applications must be routed simultaneously; application k requests a nominal flow λ_k (Mbps), tolerates a maximum end-to-end delay h_k (ms) and carries a priority weight δ_k.

A. Decision Variables

Let $x_{ij}^k \in \{0, 1\}$ indicate whether route k uses link (i, j). Let $y_k \in \{0, 1\}$ indicate whether application k is served by a feasible route. Let $w_k \in \{0, 1\}$ indicate that every link along route k can carry its aggregate load, and $t_k \in \{0, 1\}$ indicate that route k satisfies its delay budget.

B. Objective Function

The objective is to *maximize* a weighted sum that rewards the number of served high-priority connections, the number of connections whose links are not overloaded, and the number of connections that meet their delay constraint [4]:

$$\mathrm{FO} \;=\; \sum_{k=1}^{L} \delta_k\, y_k \;+\; p_1 \sum_{k=1}^{L} y_k \;+\; p_2 \sum_{k=1}^{L} w_k \;+\; p_3 \sum_{k=1}^{L} t_k, \quad (1)$$

with weights $p_1 = 3$, $p_2 = 2$, $p_3 = 1$ reflecting the relative importance of established connections, bandwidth-feasible connections and delay-feasible connections, respectively. For $L = 17$ with the priorities of Table I ($\sum \delta_k = 30$), the global optimum of Eq. (1) is $\mathrm{FO}^* = 30 + 51 + 34 + 17 = 132$.

IV. METHODOLOGY

Because exhaustive search over $\prod_k |R_k|$ route combinations is intractable, we generate a tractable candidate route pool with ACO and search it with GA or DE.

A. Candidate-Route Generation (ACO / AntNet)

For each application k, an ant colony builds paths from its source to its destination under a pheromone matrix τ and heuristic η. At node i, the next hop j is chosen with probability

$$P(i, j) = \frac{\left[\tau_{ij}\right]^{\alpha} \left[\eta_{ij}\right]^{\beta}}{\sum_{l \in \mathcal{N}_i} \left[\tau_{il}\right]^{\alpha} \left[\eta_{il}\right]^{\beta}}, \quad (2)$$

where α, β weigh pheromone vs. heuristic. After a complete walk, the route is stripped of loops, its links reinforce the pheromone proportionally to the carried bandwidth, and a global evaporation rate ρ prevents premature convergence. Repeating this over many ants and iterations yields a pool R_k of diverse candidate routes per application. On the studied topology this produced hundreds of candidate routes.

B. Route-Set Selection with GA and DE

Let $m = \sum_k |R_k|$ be the total number of candidate routes and let each individual be a vector $\mathbf{s} \in \mathbb{Z}^L$ selecting one route per application (s_k restricted to R_k). Both GA and DE seek $\max_{\mathbf{s}} \mathrm{FO}(\mathbf{s})$.

a) Geneetic Algorithm.: The GA keeps a population of route-set vectors, applies roulette-wheel parent selection with pressure $\beta_s = 8$, arithmetic crossover ($\gamma = 0.4$) and Gaussian mutation ($\sigma = 0.1 \cdot m$), merges parents and offspring, sorts by fitness and truncates to the population size. Crossover and mutation operators clamp every s_k to the valid range of application k, so each selected route always belongs to its own application.

b) Differential Evolution.: The DE implementation uses the rand/1 mutation

$$y = \mathbf{s}_{r_1} + \beta\big(\mathbf{s}_{r_2} - \mathbf{s}_{r_3}\big), \quad (3)$$

with $\beta \sim \mathcal{U}(0.2, 0.8)$, rounded to integers and clamped per application, followed by a binomial crossover with probability p_{CR} and a greedy selection that keeps the better of parent/trial.

Algorithm 1 Hybrid route optimization (ACO+GA shown; ACO+DE analogous).

Input: topology, application demand table
Run ACO (Eq. 2) $\Rightarrow$ route pool $R_1, \ldots, R_L$
Initialize GA population of route-set vectors (per-app clamped)
repeat
  Evaluate fitness $\mathrm{FO}(\mathbf{s})$ (Eq. 1)
  Parent selection (roulette) and arithmetic crossover
  Gaussian mutation; merge; sort by fitness; truncate
until stopping criterion (300 generations)
Return best route set $\mathbf{s}^*$

C. Dijkstra Baseline

Each application k is routed along the Euclidean shortest path from its source to its destination (hop-minimizing on the spatial layout). This is the classical routing baseline against which the optimized networks are compared, and it coincides with the routing routinely installed by distributed shortest-path protocols.

V. Network Emulation

To quantify the attainable QoS gains, the two competing configurations (optimized vs. Dijkstra) are embedded in a discrete-event network simulator built with `simpy`. Every link transmits packets of 512 bytes at a nominal capacity of 2 Mbps with the per-link delay matrix c_{ij}; each application injects packets at a constant rate λ_k derived from its demand, and each packet is forwarded along its route through the switches. Links employ a serialized bounded FIFO queue; when the queue is full the packet is dropped. For each configuration the simulator records received throughput, end-to-end delay, jitter (std. of the delay) and packet-loss ratio under identical traffic.

VI. Results and Discussion

A. Objective-Function Scores

Table III reports the objective-function value reached by each method on the 52-node / 17-application instance. Both hybrids attain the global optimum $\mathrm{FO}^* = 132$, while the Euclidean-shortest (Dijkstra) routing, which ignores link contention, only reaches 120. The GA slightly edges the DE on this instance, but both confirm that the hybrid search is able to find a route set in which no link is overloaded and every delay budget is respected — something the shortest-path baseline fails to do.

TABLE II
Network-level results (optimized ACO+GA vs. Dijkstra, same traffic).

Metric	Optimized	Dijkstra	Improvement
Throughput (Mbps)	0.560	0.408	$+37\%$
End-to-end delay (ms)	115	198	$\approx 1.7\times$ lower
Jitter (ms)	0.74	33.2	$\approx 45\times$ lower
Packet loss	2.3%	30.8%	$\approx 13\times$ lower

TABLE III
Objective-function scores (optimum = 132).

Method	Fitness	Optimal?
Euclidean shortest path (Dijkstra)	120	No
Random route set	112	No
ACO + DE	128	Near
ACO + GA	**132**	**Yes**

B. Network Performance

Table II shows the aggregated network metrics of the two simulated SDN configurations. The optimized network achieves markedly better QoS across all four dimensions studied: it delivers $+37\%$ throughput, and its end-to-end delay, jitter and packet loss are all substantially lower. The largest relative gains are observed for delay-sensitive flows (video/voice) that, in the Dijkstra configuration, are concentrated on a few over-subscribed core links (e.g., the 14–15–21–22 corridor): these applications suffer 57–94% packet loss with Dijkstra but only 1–3% with the optimized routes. This matches the qualitative behavior reported by previous SDN-TE studies and confirms that balancing the offered load across the core – rather than minimizing each path in isolation – is what protects QoS.

C. Discussion

a) Why shortest path is not enough.: Dijkstra minimizes per-application path length but ignores that many flows may share the same bottleneck links. On this topology the shortest-path assignment overloads several core links (utilization up to $\approx 2\times$ their capacity), producing congestion, queue growth and drops. The objective function of Eq. (1) explicitly penalizes overloaded links and delay violations, which drives the hybrids toward a *balanced, feasible* routing.

b) Generality.: The two-stage scheme is topology-agnostic: ACO supplies diversity, and the selection stage enforces global feasibility. It extends naturally to larger networks, online re-routing and additional constraints (e.g., link utilization, energy, path diversity).

VII. Conclusion and Future Work

This paper showed that traffic-engineering optimization is a powerful and economically attractive complement to capacity expansion in SDNs. Two hybrid metaheuristics (ACO+GA and ACO+DE) were formulated, implemented and evaluated on a realistic 52-node topology with 17 QoS-constrained flows. Both reached the global optimum of the proposed objective function, and discrete-event emulation showed the optimized SDN outperforming a Euclidean-shortest-path baseline by $+37\%$ in throughput while reducing delay, jitter and loss by one to two orders of magnitude.

Future work will: (i) scale the evaluation to larger and publicly available topologies; (ii) integrate the optimizer with an online SDN controller for dynamic re-routing; (iii) include additional QoS metrics (energy, reliability) and multi-objective formulations; and (iv) strengthen the statistical analysis with more realizations and confidence intervals.

REFERENCES

[1] International Telecommunication Union, "ICT facts and figures 2017," Geneva, 2017.
[2] H. Kim and N. Feamster, "Improving network management with software defined networking," *IEEE Commun. Mag.*, 2013.
[3] F. Hu, Q. Hao and M. Bao, "A survey on software-defined network and OpenFlow: from concept to implementation," *Commun. Surveys & Tutorials*, 2014.
[4] — , "Traffic engineering QoS model (adapted)," 2004.
[5] M. Dorigo and T. Stützle, *Ant Colony Optimization*. MIT Press, 2004.
[6] S. M. K. Heris, "Differential Evolution in MATLAB," MathWorks File Exchange, 2016.
[7] S. M. K. Heris, "Genetic Algorithm in MATLAB (YPEA101)," MathWorks File Exchange, 2016.
[8] J. Kirk, "dijkstra.m," MathWorks File Exchange, 2015.
[9] V. Lakshminarayan and T. Maier, "SDN traffic engineering," 2006.